BitFlip

CloudSherpa

Software Requirements Specification

Contents

1 Introduction

CloudSherpa is a multi-cloud cost management and optimization platform designed to address the growing complexity of managing cloud infrastructure across AWS, Azure, and Google Cloud Platform.

As organizations increasingly adopt multi-cloud strategies, they face challenges in monitoring spending, identifying cost anomalies, and optimizing resource utilization across fragmented platforms. CloudSherpa consolidates billing and usage data from all three major cloud providers into a unified dashboard, providing real-time cost visibility, predictive forecasting, and actionable optimization recommendations.

The platform enables teams to make data-driven decisions, reduce cloud expenditure, and improve resource efficiency through comprehensive analytics, automated alerts, and scenario planning capabilities.

2 User Stories

2.1 Connect AWS Account

User Story

As a platform user, I want to connect my AWS account so that CloudSherpa can collect its usage, billing, and performance data.

Acceptance Criterion

Given valid AWS credentials, when the user connects the account, then CloudSherpa stores the connection and successfully imports AWS usage, billing, and performance data.

Definition of Done

- AWS account connection flow is implemented end to end.
- Valid AWS credentials result in successful connection and data import.

- Failed AWS connection scenarios are handled and tested.
- Relevant documentation/configuration steps are updated.

Positive Test Case

Enter valid AWS credentials and submit the connection form, verify the AWS account is linked and usage, billing, and performance data are imported.

Negative Test Case

Enter invalid or expired AWS credentials and submit, verify the connection is rejected, no account is linked, and a clear error message is shown.

2.2 Connect Azure Account

User Story

As a platform user, I want to connect my Azure account so that I can monitor Azure spending alongside other providers.

Acceptance Criterion

Given valid Azure credentials, when the user connects the account, then CloudSherpa stores the connection and imports Azure usage, billing, and performance data.

Definition of Done

- Azure account connection flow is implemented end to end.
- Valid Azure credentials result in successful connection and data import.
- Failed Azure connection scenarios are handled and tested.
- Relevant documentation/configuration steps are updated.

Positive Test Case

Enter valid Azure credentials and connect the account, verify Azure data is successfully imported and displayed.

Negative Test Case

Enter invalid Azure tenant or credential details, verify the connection fails, no data is imported, and the user receives an error message.

2.3 Connect GCP Account

User Story

As a platform user, I want to connect my GCP account so that I can consolidate GCP metrics into one platform.

Acceptance Criterion

Given valid GCP credentials, when the user connects the account, then CloudSherpa stores the connection and imports GCP usage, billing, and performance data.

Definition of Done

- GCP account connection flow is implemented end to end.
- Valid GCP credentials result in successful connection and data import.
- Failed GCP connection scenarios are handled and tested.
- Relevant documentation/configuration steps are updated.

Positive Test Case

Upload valid GCP service account credentials and connect, verify the GCP account is linked and relevant data is imported.

Negative Test Case

Upload malformed or unauthorized GCP credentials, verify the system blocks the connection and displays an actionable error.

2.4 Automatic Ingestion Jobs

User Story

As a platform administrator, I want provider-specific ingestion jobs to run automatically so that data remains up to date without manual effort.

Acceptance Criterion

Given a connected cloud account, when the scheduled ingestion time is reached, then the relevant ingestion job runs automatically and refreshes the provider data.

Definition of Done

- Automatic ingestion scheduling is implemented for connected providers.
- Scheduled jobs run without manual intervention.

- Job success/failure is logged and testable.
- Latest provider data is refreshed after job execution.

Positive Test Case

Configure a connected provider and wait for the scheduled ingestion time, verify the ingestion job runs automatically and updates provider data.

Negative Test Case

Disable or misconfigure the scheduler, verify the ingestion job does not run and the failure is logged or flagged for review.

2.5 Clear Feedback on Failed Integrations

User Story

As a platform user, I want failed cloud integrations to show clear error feedback so that I can fix configuration issues quickly.

Acceptance Criterion

Given a failed integration attempt, when the failure occurs, then the system displays a clear error message indicating that the connection failed and why.

Definition of Done

- Integration failure messages are implemented for all supported providers.
- Error messages are clear, user-facing, and actionable.
- Common failure cases are covered by tests.
- UI displays the failure state correctly.
- Error handling behavior is reviewed and accepted.

Positive Test Case

Trigger a known connection failure, such as missing permissions, verify the system displays the correct provider-specific failure reason.

Negative Test Case

Trigger an unexpected backend error, verify the system does not crash and still shows a generic but useful failure message.

2.6 Store Historical Cloud Data

User Story

As a platform user, I want historical cloud data to be stored so that I can analyze cost and usage trends over time.

Acceptance Criterion

Given imported cloud data, when ingestion completes, then the system stores the historical records for later retrieval and analysis.

Definition of Done

- Historical data persistence is implemented.
- Ingested records are retrievable after storage.
- Data retention is verified through tests.
- Storage behavior is documented.

Positive Test Case

Complete a successful ingestion cycle, verify historical records are stored and can be retrieved later.

Negative Test Case

Simulate a database write failure during ingestion, verify the system does not falsely report success and the failure is logged.

2.7 Normalize Incoming Provider Data

User Story

As a data processing service, I want incoming provider data to be normalized so that analytics can be performed consistently across clouds.

Acceptance Criterion

Given raw provider data from AWS, Azure, or GCP, when it is processed, then it is transformed into a common internal format.

Definition of Done

- Normalization logic is implemented for AWS, Azure, and GCP inputs.
- Provider data maps into a common internal schema.

- Schema transformation is covered by automated tests.
- Invalid or incomplete input is handled safely.

Positive Test Case

Send valid AWS, Azure, and GCP raw data into the processing pipeline, verify each is transformed into the common internal schema.

Negative Test Case

Send incomplete or corrupted provider data, verify invalid fields are rejected or flagged without corrupting the normalized dataset.

2.8 Aggregate Time-Series Data

User Story

As a platform user, I want time-series data to be aggregated by period so that I can view daily, weekly, and monthly insights.

Acceptance Criterion

Given stored cloud data, when the user selects a time interval, then the system returns aggregated results for that interval.

Definition of Done

- Aggregation logic exists for daily, weekly, and monthly intervals.
- User-selected intervals return correct aggregated values.
- Aggregated data is consumable by dashboard/reporting features.
- Edge cases such as empty datasets are handled.

Positive Test Case

Request daily, weekly, and monthly views for a valid dataset, verify the returned values match expected aggregates.

Negative Test Case

Request aggregation for an empty or invalid date range, verify the system returns an empty state or validation error without failing.

2.9 Reliable Processing Pipeline

User Story

As a system administrator, I want processed data pipelines to be reliable so that dashboards and services reflect trusted information.

Acceptance Criterion

Given incoming data for processing, when the pipeline executes, then the system completes processing without data loss and flags failed jobs for review.

Definition of Done

- Processing pipeline is implemented with failure handling.
- Failed jobs are flagged or logged for review.
- No data loss occurs in tested normal and failure scenarios.
- Pipeline reliability is covered by automated tests.

Positive Test Case

Process a full batch of valid provider data, verify the pipeline completes successfully without data loss.

Negative Test Case

Force a mid-pipeline service interruption, verify failed jobs are flagged and partial data is not silently accepted as complete.

2.10 Forecast Future Cloud Costs

User Story

As a finance-focused user, I want forecasted cloud costs for upcoming periods so that I can plan budgets proactively.

Acceptance Criterion

Given sufficient historical cost data, when the user requests a forecast, then the system displays predicted cloud costs for a future period.

Definition of Done

- Forecasting feature is implemented and accessible to users.
- Sufficient historical data triggers forecast generation.

- Forecast results display correctly for the requested period.
- Forecast generation is covered by automated tests.

Positive Test Case

Provide sufficient historical cost data and request a forecast, verify the system generates a forecast for the selected future period.

Negative Test Case

Request a forecast when insufficient historical data exists, verify the system does not generate misleading output and shows an appropriate message.

2.11 View Spending Trends Over Time

User Story

As a platform user, I want to see spending trends over time so that I can identify whether my costs are rising or falling.

Acceptance Criterion

Given historical spending data, when the user views trends, then the system displays the change in cost over time.

Definition of Done

- Trend visualization over time is implemented.
- Rising/falling cost movement is clearly shown.
- Trend calculations are tested with sample datasets.
- UI labeling is understandable and verified.

Positive Test Case

Open the trend view for a dataset with changing costs, verify the system correctly displays rising or falling trends.

Negative Test Case

Use a dataset with missing periods or invalid timestamps, verify the system handles gaps correctly and does not produce incorrect trend visuals.

2.12 Forecast Per Provider

User Story

As an operations manager, I want forecasts per provider so that I can understand where future spend will be concentrated.

Acceptance Criterion

Given connected providers with historical data, when the user requests provider forecasts, then the system displays separate forecasts for each provider.

Definition of Done

- Per-provider forecasting is implemented.
- Forecast output is separated by provider.
- Provider-level forecast calculations are tested.
- UI supports viewing each provider forecast clearly.

Positive Test Case

Connect multiple providers with historical data and request provider forecasts, verify separate forecasts are shown for each provider.

Negative Test Case

Request a per-provider forecast when one provider has no usable data, verify only valid providers are forecasted and the missing provider is flagged.

2.13 Surface Unusual Trend Shifts

User Story

As a platform user, I want the system to surface unusual trend shifts so that I can investigate them early.

Acceptance Criterion

Given forecast and historical trend data, when a significant deviation is detected, then the system flags the shift as unusual.

Definition of Done

- Unusual trend-shift detection logic is implemented.
- Significant deviations are flagged in the interface.

- Detection thresholds/rules are configurable or documented.
- Detection behavior is tested with abnormal sample data.

Positive Test Case

Load data with a sharp cost spike beyond the defined threshold, verify the system flags it as unusual.

Negative Test Case

Load data with minor normal fluctuations, verify the system does not incorrectly flag them as anomalies.

2.14 Identify Idle Resources

User Story

As a platform user, I want the system to identify idle resources so that I can reduce unnecessary cloud spend.

Acceptance Criterion

Given utilization data, when resources fall below the configured activity threshold, then the system marks them as idle.

Definition of Done

- Idle resource detection logic is implemented.
- Threshold-based idle classification works on real/sample data.
- Detected idle resources are visible to the user.
- Detection logic is covered by automated tests.

Positive Test Case

Provide utilization data for a resource below the idle threshold, verify the system marks it as idle.

Negative Test Case

Provide utilization data just above the threshold, verify the system does not incorrectly classify the resource as idle.

2.15 Recommend Right-Sizing for Overprovisioned Resources

User Story

As a platform user, I want recommendations for overprovisioned resources so that I can right-size infrastructure.

Acceptance Criterion

Given resource utilization data, when sustained underuse is detected, then the system provides a right-sizing recommendation.

Definition of Done

- Overprovisioning detection and right-sizing recommendation logic is implemented.
- Underused resources generate recommendations correctly.
- Recommendation rules are tested with sample cases.
- Recommendations are displayed clearly in the UI.

Positive Test Case

Provide sustained low-utilization metrics for a large resource, verify the system generates a right-sizing recommendation.

Negative Test Case

Provide variable or high utilization metrics, verify the system does not recommend right-sizing inappropriately.

2.16 Show Estimated Savings

User Story

As a technical decision-maker, I want each recommendation to include estimated savings so that I can prioritize actions.

Acceptance Criterion

Given a generated optimization recommendation, when it is displayed, then it includes an estimated cost-saving value.

Definition of Done

- Estimated savings are calculated for each recommendation.

- Savings values are displayed with the recommendation.
- Savings calculations are tested for correctness.
- Missing/insufficient data cases are handled gracefully.

Positive Test Case

Open a valid optimization recommendation, verify estimated cost savings are displayed.

Negative Test Case

Use a recommendation with insufficient pricing data, verify the system does not display an incorrect savings value and instead shows an unavailable state.

2.17 Group Recommendations by Severity

User Story

As a platform user, I want recommendations to be grouped by severity so that I can focus on the highest-value optimizations first.

Acceptance Criterion

Given multiple optimization recommendations, when they are displayed, then the system groups or sorts them by severity.

Definition of Done

- Recommendation grouping/sorting by severity.
- Multiple recommendations appear in prioritized order.
- Sorting/grouping logic is tested.
- UI makes prioritization clear to the user.

Positive Test Case

Generate multiple recommendations with different savings values, verify they are grouped or sorted correctly.

Negative Test Case

Include recommendations with identical or missing severity data, verify the UI still behaves predictably and does not crash or misorder unpredictably.

2.18 Dashboard Overview of Usage and Spend

User Story

As a platform user, I want a dashboard overview of cloud usage and spend so that I can understand my environment at a glance.

Acceptance Criterion

Given connected cloud accounts with available data, when the user opens the dashboard, then the dashboard shows a summary of usage and spend.

Definition of Done

- Dashboard overview page is implemented.
- Usage and spend summary cards/widgets display correctly.
- Dashboard loads connected account data successfully.
- Empty and populated states are handled.

Positive Test Case

Log in with connected accounts containing data, verify the dashboard displays an overview of usage and spend.

Negative Test Case

Log in with no connected accounts or no data, verify the dashboard shows an appropriate empty state instead of broken widgets.

2.19 Visual Charts for Forecasts and Trends

User Story

As a platform user, I want visual charts for forecasts and trends so that I can interpret data quickly.

Acceptance Criterion

Given forecast and trend data, when the user navigates to the relevant view, then the system displays the data in chart form.

Definition of Done

- Forecast and trend charts are implemented.
- Charts render accurate data from the backend.

- Chart interactions/labels are tested and readable.
- Failure or no-data states are handled.

Positive Test Case

Navigate to the analytics section with available trend and forecast data, verify charts render correctly.

Negative Test Case

Simulate backend chart-data failure, verify the UI shows an error or fallback state rather than broken visuals.

2.20 Display Recommendations and Anomalies on Dashboard

User Story

As a platform user, I want recommendations and anomalies displayed in the dashboard so that I can take action from one place.

Acceptance Criterion

Given detected recommendations or anomalies, when the dashboard loads, then those items are visible in a dedicated dashboard section.

Definition of Done

- Dashboard sections for recommendations and anomalies are implemented.
- Backend data feeds populate these sections correctly.
- Items are visible on dashboard load.
- UI and backend integration are tested.

Positive Test Case

Generate anomalies and recommendations in the backend, verify both appear in their dashboard sections.

Negative Test Case

Remove all anomalies and recommendations, verify the dashboard correctly shows that there are no actionable items.

2.21 Clear Presentation for Non-Technical Users

User Story

As a non-technical user, I want the dashboard to present information clearly so that I can still make informed decisions.

Acceptance Criterion

Given dashboard data is displayed, when a non-technical user views it, then the main metrics and statuses are labeled in plain, understandable language.

Definition of Done

- Dashboard wording and labels are understandable to non-technical users.
- Key metrics are presented without unnecessary jargon.
- UI review confirms clarity and accessibility.

Positive Test Case

Review the dashboard labels and summaries as a non-technical user, verify key metrics are understandable without deep cloud knowledge.

Negative Test Case

Inject highly technical raw provider terms into the display, verify the system either maps them to plain language or identifies the issue for correction.

2.22 Secure Registration and Login

User Story

As a new user, I want to create an account and log in securely so that I can access CloudSherpa safely.

Acceptance Criterion

Given a new user submits valid registration details, when registration is completed, then the user can log in and access their account securely.

Definition of Done

- User registration and secure login are implemented.
- Valid registration creates an account successfully.
- Authentication flow is tested for valid/invalid cases.

- Sensitive authentication data is handled securely.

Positive Test Case

Register with valid details and log in with the new account, verify access is granted securely.

Negative Test Case

Attempt login with an incorrect password, verify access is denied and no session is created.

2.23 Link Multiple Cloud Environments

User Story

As a platform user, I want to link multiple cloud environments to my profile so that I can manage them from one place.

Acceptance Criterion

Given a logged-in user, when they add more than one cloud environment, then all linked environments appear under their profile.

Definition of Done

- Users can link multiple cloud environments to one profile.
- Linked environments persist and display correctly.
- Add/remove/view environment flows are tested.
- UI reflects multiple linked environments clearly.

Positive Test Case

Add multiple valid cloud environments to one user profile, verify all appear and remain accessible.

Negative Test Case

Attempt to add a duplicate or invalid environment, verify the system rejects it and preserves existing links.

2.24 Role-Based Access Control

User Story

As an administrator, I want role-based access control so that users only see what they are authorized to access.

Acceptance Criterion

Given users with different roles, when they access the system, then each user can only access the features and data allowed for their role.

Definition of Done

- Role-based access control is implemented.
- Different roles see only allowed features/data.
- Access restrictions are covered by automated tests.
- Unauthorized access attempts are blocked correctly.

Positive Test Case

Log in as users with different roles, verify each only sees permitted features and data.

Negative Test Case

Attempt to access an admin-only page using a lower-privileged account, verify access is blocked.

2.25 Revoke Connected Cloud Accounts

User Story

As a platform user, I want to manage and revoke connected cloud accounts so that I stay in control of my integrations.

Acceptance Criterion

Given a connected cloud account, when the user revokes or removes it, then the integration is disabled and no further data is ingested from it.

Definition of Done

- Users can revoke/remove connected cloud accounts.
- Revoked integrations stop further ingestion.
- Revocation state is persisted and reflected in UI.
- Removal/revocation flows are tested.

Positive Test Case

Revoke a connected cloud integration, verify it is removed or disabled and future ingestion stops.

Negative Test Case

Attempt to revoke an integration the user does not own, verify the action is denied.

2.26 Alerts for Anomalous Spending

User Story

As a platform user, I want to receive alerts when anomalous spending is detected so that I can respond quickly.

Acceptance Criterion

Given anomalous spending is detected, when the alert condition is met, then the system sends an anomaly notification to the user.

Definition of Done

- Anomalous spending alert generation is implemented.
- Alert triggers when anomaly conditions are met.
- Alert content reaches the user successfully.
- Alert trigger logic is tested with sample anomalies.

Positive Test Case

Create spending behavior that exceeds anomaly criteria, verify an anomaly alert is sent.

Negative Test Case

Use normal spending patterns, verify no anomaly alert is triggered.

2.27 Budget Threshold Alerts

User Story

As a platform user, I want to define budget thresholds so that I am notified before overspending becomes severe.

Acceptance Criterion

Given a user-configured budget threshold, when spending reaches or exceeds that threshold, then the system sends a budget alert.

Definition of Done

- Users can configure budget thresholds.
- Threshold crossings trigger alerts correctly.
- Threshold values persist and can be edited.
- Alert triggering is covered by automated tests.

Positive Test Case

Set a budget threshold and exceed it, verify the system sends a threshold alert.

Negative Test Case

Set a threshold but keep spending below it, verify no budget alert is sent.

2.28 Notifications for Optimization Opportunities

User Story

As a platform user, I want optimization opportunities to trigger notifications so that I do not miss savings.

Acceptance Criterion

Given a new optimization opportunity is identified, when it is generated, then the system sends a notification to the user.

Definition of Done

- Optimization opportunity notifications are implemented.
- New opportunities trigger notifications automatically.
- Notification content identifies the opportunity clearly.
- Trigger logic is tested.

Positive Test Case

Generate a new optimization opportunity, verify the user receives a notification.

Negative Test Case

Reprocess unchanged data with no new opportunities, verify duplicate or unnecessary notifications are not sent.

2.29 Deliver Alerts Through Selected Channel

User Story

As a platform user, I want alerts delivered through channels like email or SMS so that I can receive them where convenient.

Acceptance Criterion

Given the user has selected a notification channel, when an alert is triggered, then the system delivers it through the selected channel.

Definition of Done

- Notification channel selection is implemented.
- Alerts are delivered through the selected channel.
- Channel-specific delivery is tested.
- Failure to deliver is logged or surfaced appropriately.

Positive Test Case

Select email as the preferred notification channel and trigger an alert, verify the alert is delivered by email.

Negative Test Case

Select an unavailable or invalid channel configuration, such as an unverified phone number, verify delivery fails gracefully and the issue is logged or shown.

2.30 Downloadable Reports

User Story

As a manager, I want downloadable reports so that I can share cloud-cost insights with stakeholders.

Acceptance Criterion

Given report data is available, when the user chooses to download a report, then the system generates and downloads the report file.

Definition of Done

- Downloadable report generation is implemented.
- Generated reports include the expected data.

- Report download works in supported format(s).
- Generation and download are tested end to end.

Positive Test Case

Generate a report for a valid dataset and download it, verify the file is created and contains the correct data.

Negative Test Case

Attempt to generate a report when the data service is unavailable, verify the system shows a failure message and no corrupt file is downloaded.

2.31 Trend Analysis Summaries

User Story

As a platform user, I want trend analysis summaries so that I can understand long-term cost behavior.

Acceptance Criterion

Given historical data exists, when the user requests a trend summary, then the system displays a summary of long-term cost behavior.

Definition of Done

- Trend analysis summaries are implemented.
- Long-term cost behavior is summarized accurately.
- Summary logic is tested with historical data.
- UI displays the summary clearly.

Positive Test Case

Request a trend summary for historical cost data, verify the summary accurately reflects the long-term pattern.

Negative Test Case

Request a summary for an empty dataset, verify the system does not fabricate a summary and instead returns an empty-state response.

2.32 Visual Summaries of Key Metrics

User Story

As an auditor or reviewer, I want visual summaries of key metrics so that I can assess cloud usage efficiently.

Acceptance Criterion

Given analytics data is available, when the user opens the analytics view, then key metrics are shown visually in summary format.

Definition of Done

- Visual metric summaries are implemented in analytics view.
- Key metrics render correctly in summary format.
- Visual summaries are tested with sample data.
- No-data/error states are handled.

Positive Test Case

Open the analytics view with valid metrics, verify visual summaries render correctly.

Negative Test Case

Provide incomplete metrics data, verify missing values are handled gracefully rather than causing broken visuals.

2.33 Filter Reports by Provider, Time Period, or Environment

User Story

As a platform user, I want reports filtered by provider, time period, or environment so that they are relevant to my use case.

Acceptance Criterion

Given report filters are available, when the user applies provider, date, or environment filters, then the generated report reflects only the filtered data.

Definition of Done

- Report filters for provider, time period, and environment are implemented.
- Applied filters affect generated output correctly.
- Filter combinations are tested.
- UI clearly shows active filters.

Positive Test Case

Apply valid filters and generate a report, verify only matching data appears in the output.

Negative Test Case

Apply conflicting or invalid filters that return no results, verify the report indicates no data rather than showing unrelated data.

2.34 Compare Costs Across AWS, Azure, and GCP

User Story

As a platform user, I want to compare costs across AWS, Azure, and GCP so that I can identify the most cost-effective provider.

Acceptance Criterion

Given cost data exists for supported providers, when the user opens the comparison view, then the system shows provider costs side by side.

Definition of Done

- Side-by-side cost comparison across supported providers is implemented.
- Comparison view displays available provider cost data correctly.
- Comparison calculations/formatting are tested.
- Empty/missing provider data is handled gracefully.

Positive Test Case

Connect all supported providers and open the cost comparison view, verify provider costs are shown side by side.

Negative Test Case

Open the comparison view with only one provider connected, verify the system does not produce a misleading multi-provider comparison.

2.35 Compare Resource Usage Across Providers

User Story

As a platform user, I want to compare resource usage across providers so that I can understand efficiency differences.

Acceptance Criterion

Given usage data exists for supported providers, when the user requests a comparison, then the system displays usage metrics side by side.

Definition of Done

- Side-by-side usage comparison across providers is implemented.
- Usage metrics render correctly for connected providers.
- Comparison view is tested with multi-provider data.
- Labels/units are clear and consistent.

Positive Test Case

Load usage metrics for multiple providers, verify the system displays side-by-side usage values.

Negative Test Case

Provide metrics in inconsistent units without normalization, verify the system blocks or normalizes the comparison rather than displaying inaccurate data.

2.36 Side-by-Side Provider Views for Technical Leads

User Story

As a technical lead, I want side-by-side provider views so that I can support architecture and deployment decisions.

Acceptance Criterion

Given multiple providers are connected, when the user views provider details, then the system presents comparable provider information in one view.

Definition of Done

- Unified provider comparison view is implemented for technical users.
- Comparable provider details are shown in one place.
- Cross-provider display logic is tested.
- UI supports architecture/deployment evaluation.

Positive Test Case

View the provider comparison page with valid multi-provider data, verify comparable details are visible in one unified view.

Negative Test Case

Attempt to access comparison details for a provider with missing permissions or unavailable data, verify that provider section is restricted or marked unavailable.

2.37 Comparison Summaries Over Time

User Story

As a manager, I want comparison summaries over time so that I can track which providers are becoming more or less economical.

Acceptance Criterion

Given historical provider data exists, when the user views comparison history, then the system shows how provider cost efficiency changes over time.

Definition of Done

- Historical comparison summary over time is implemented.
- Efficiency/cost changes across providers are visible by time period.
- Trend comparison calculations are tested.
- Visualization is clear and interpretable.

Positive Test Case

Load historical data for multiple providers and view comparison history, verify the system shows how cost efficiency changes over time.

Negative Test Case

Use a date range outside available history, verify the system returns no data or a warning instead of incorrect trends.

3 Domain Model

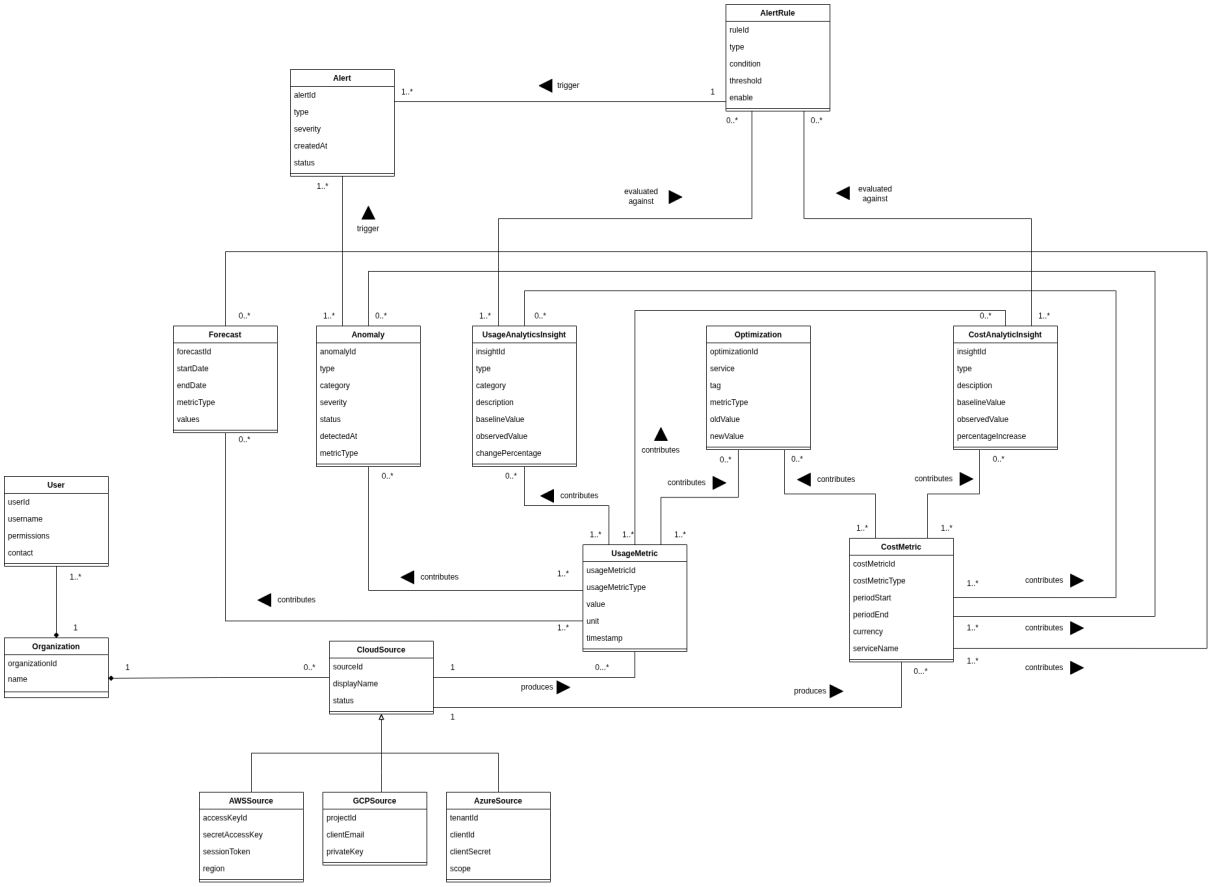
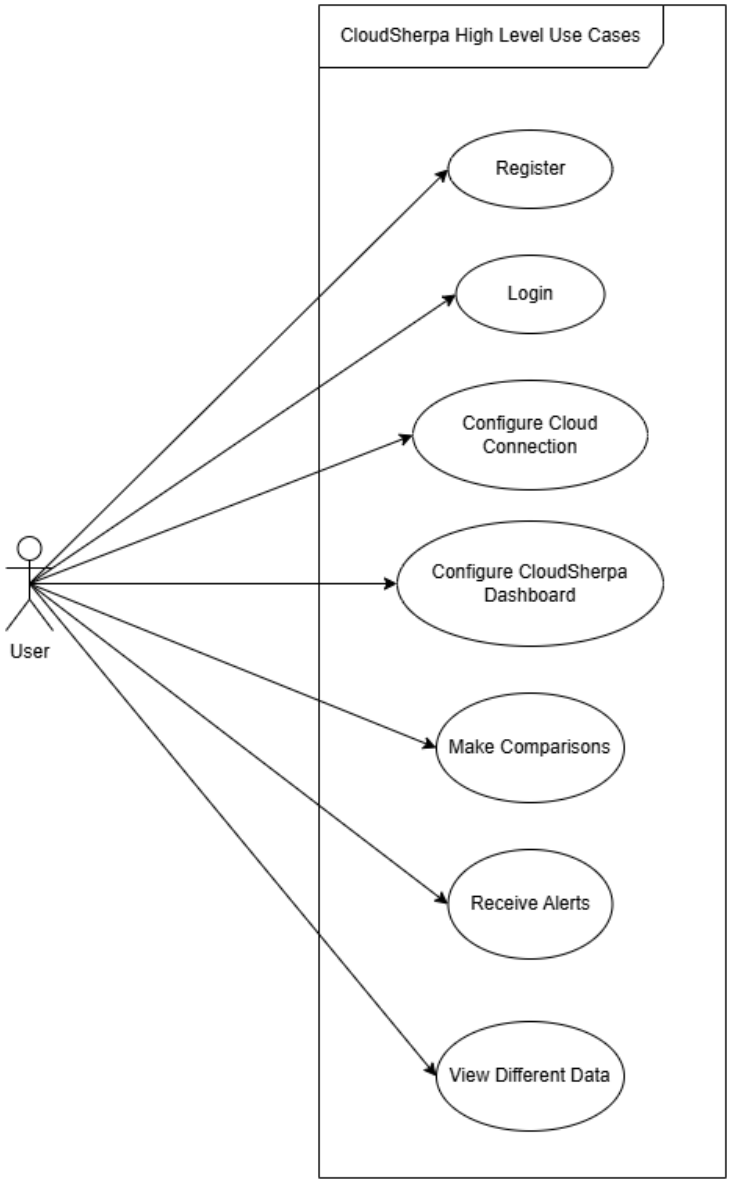


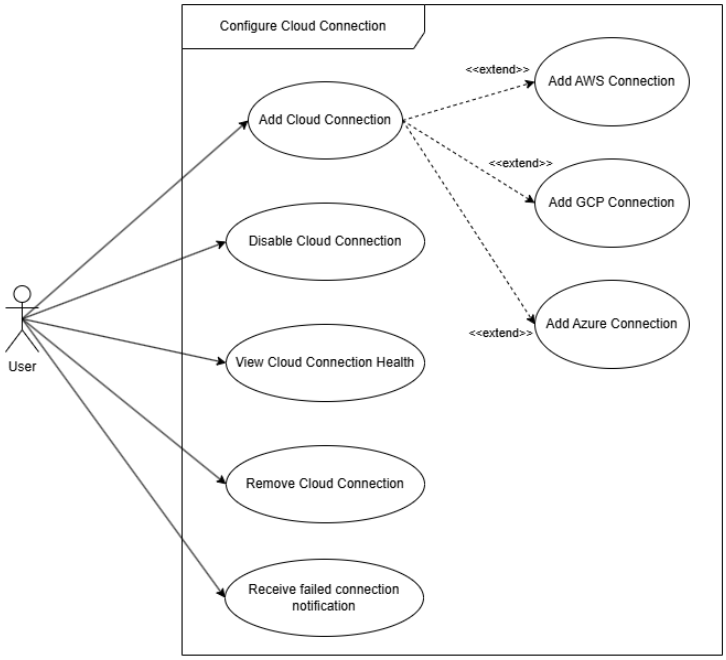
Figure 1: CloudSherpa Domain Model

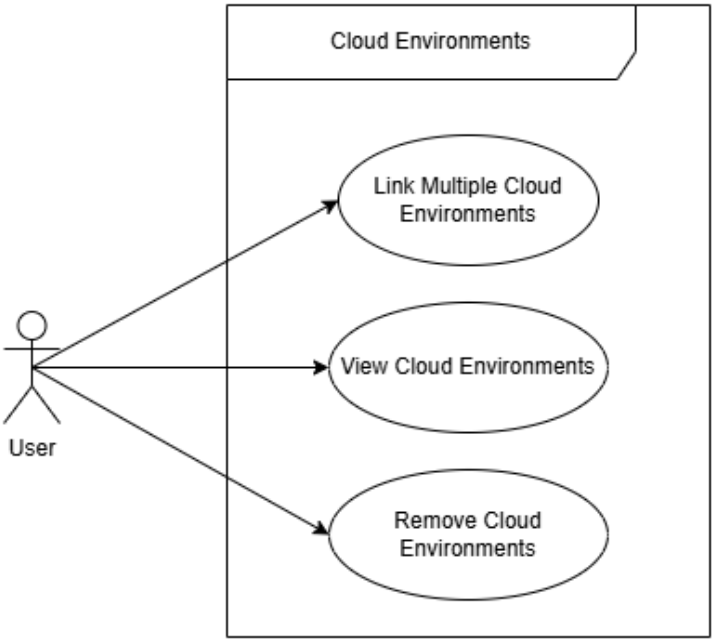
4 Use Case Diagrams

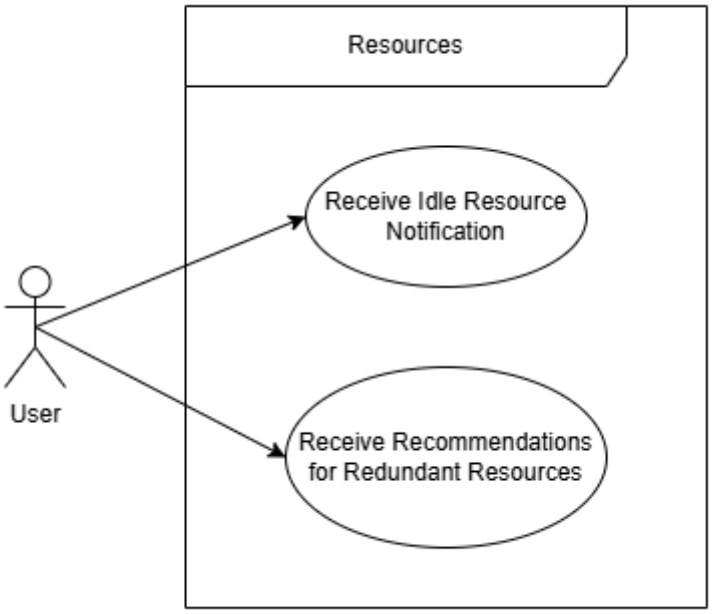
4.1 High-Level Use Case Diagram

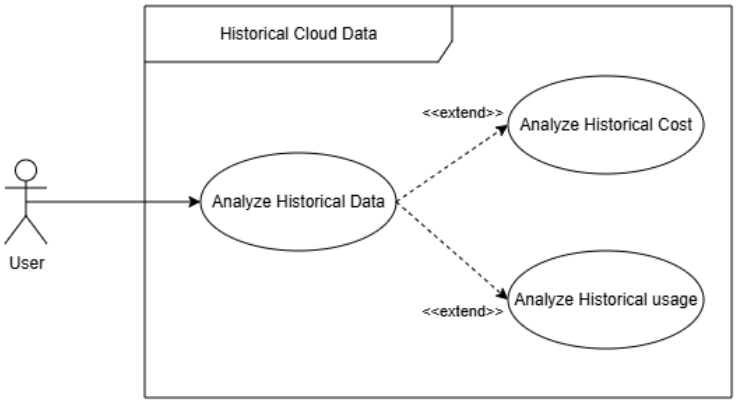


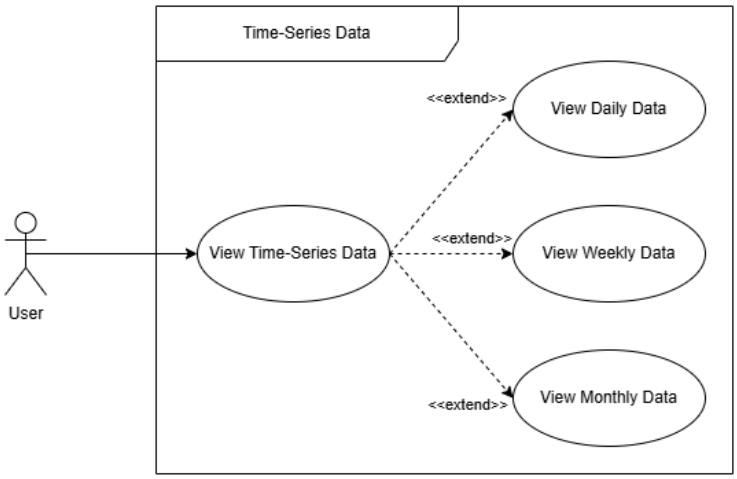
4.2 Individual Use Case Diagrams

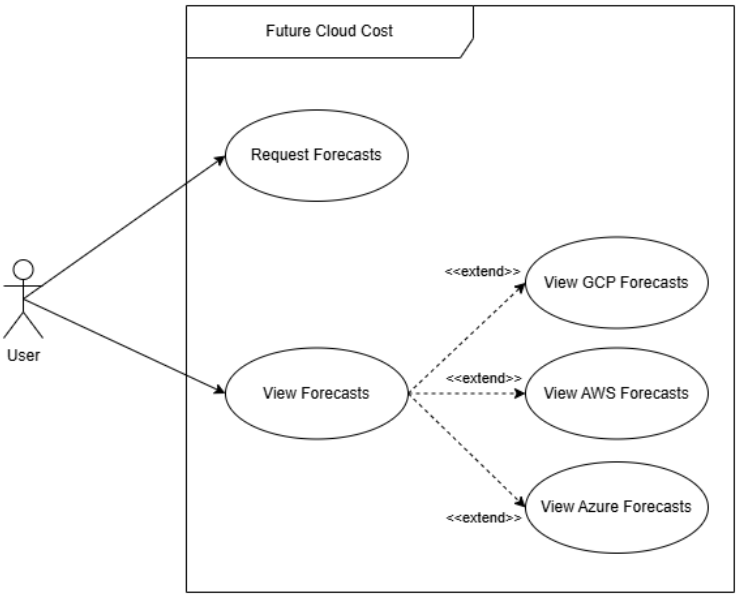


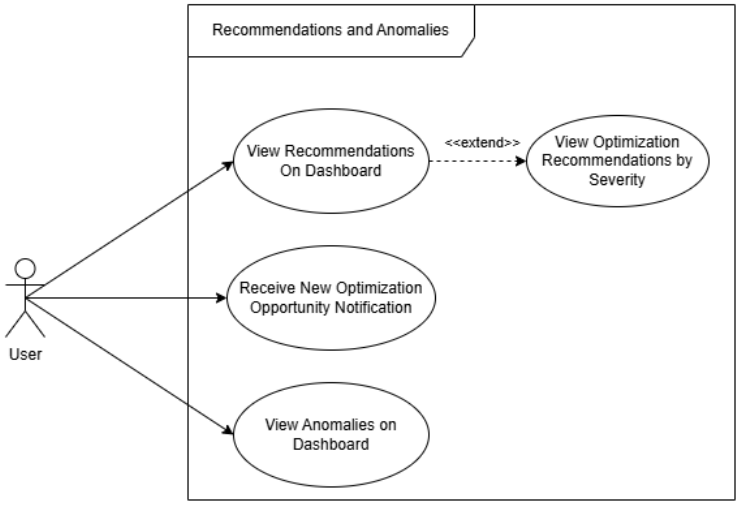


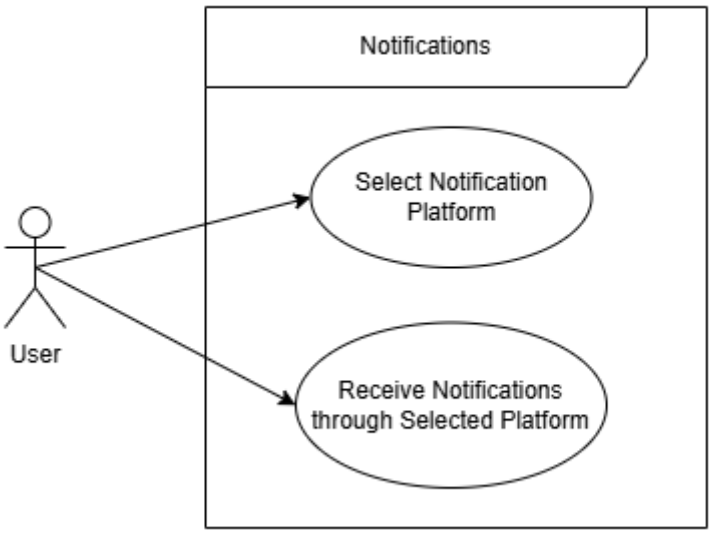

Resources
Receive Idle Resource Notification
User
Receive Recommendations for Redundant Resources

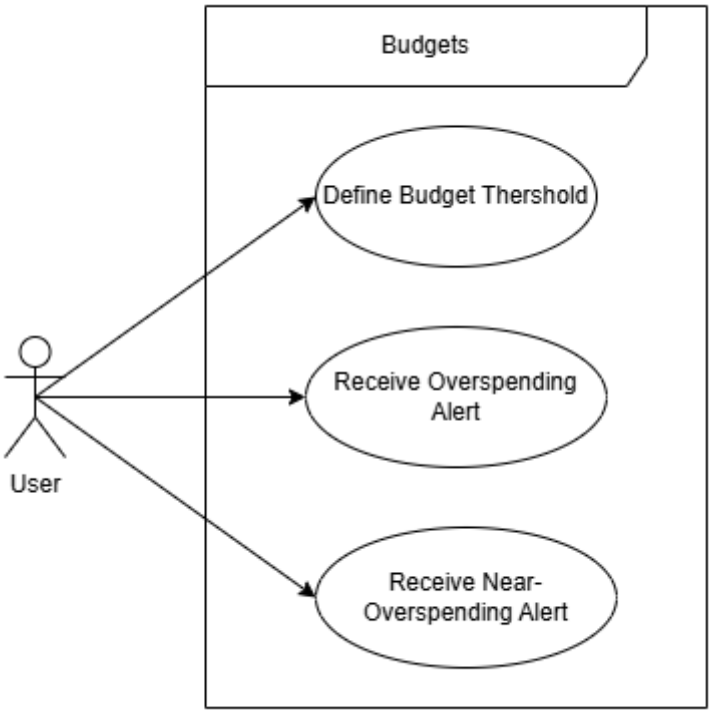

Historical Cloud Data
<<extend>>
Analyze Historical Cost
Analyze Historical Data
User
<<extend>>
Analyze Historical usage

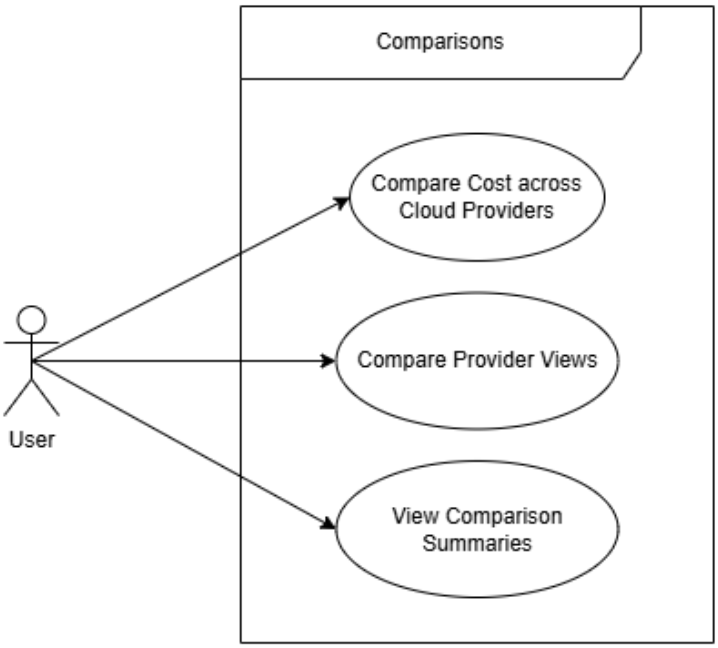

Time-Series Data
View Daily Data
<<extend>>
View Time-Series Data
<<extend>>
View Weekly Data
<<extend>>
View Monthly Data
User

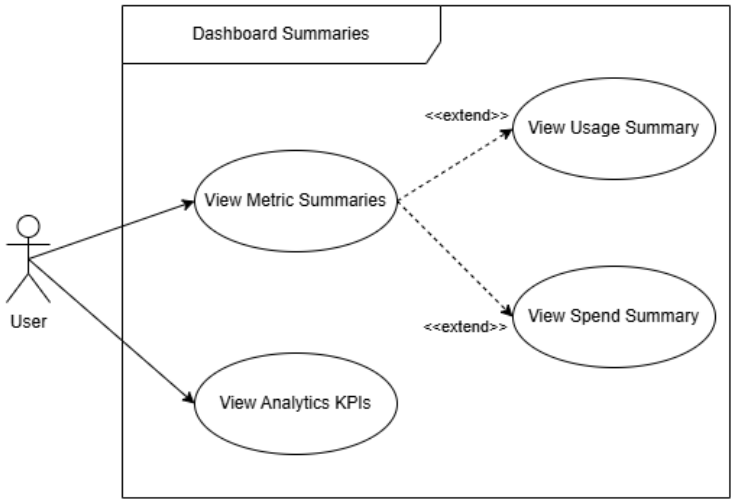

Future Cloud Cost
Request Forecasts
View GCP Forecasts
<<extend>>
View Forecasts
<<extend>>
View AWS Forecasts
<<extend>>
View Azure Forecasts
User

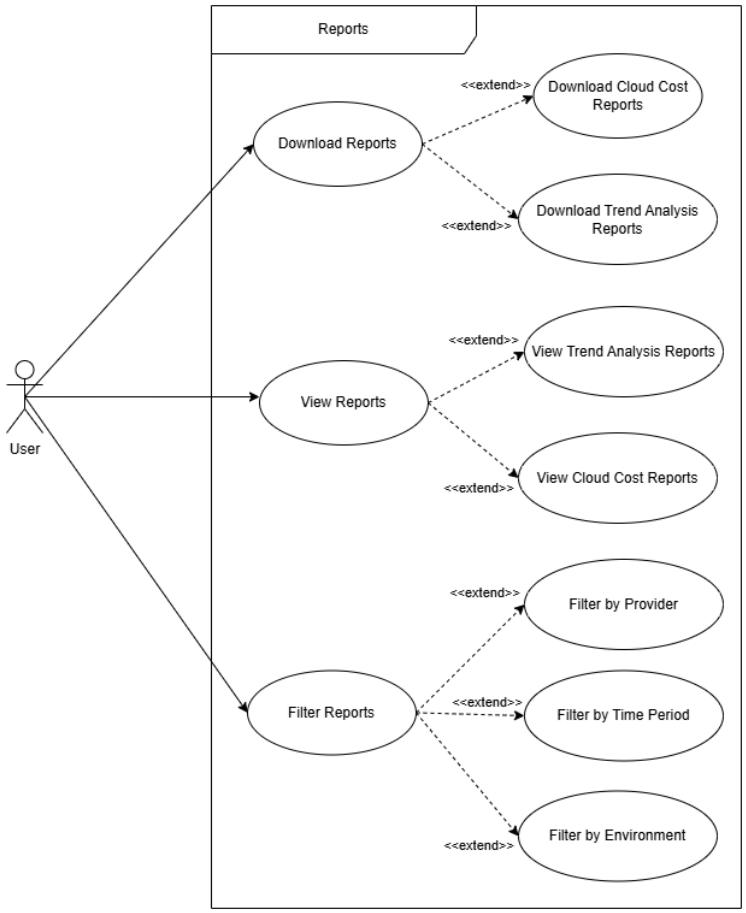

Recommendations and Anomalies
View Recommendations On Dashboard
<<extend>>
View Optimization Recommendations by Severity
Receive New Optimization Opportunity Notification
View Anomalies on Dashboard
User

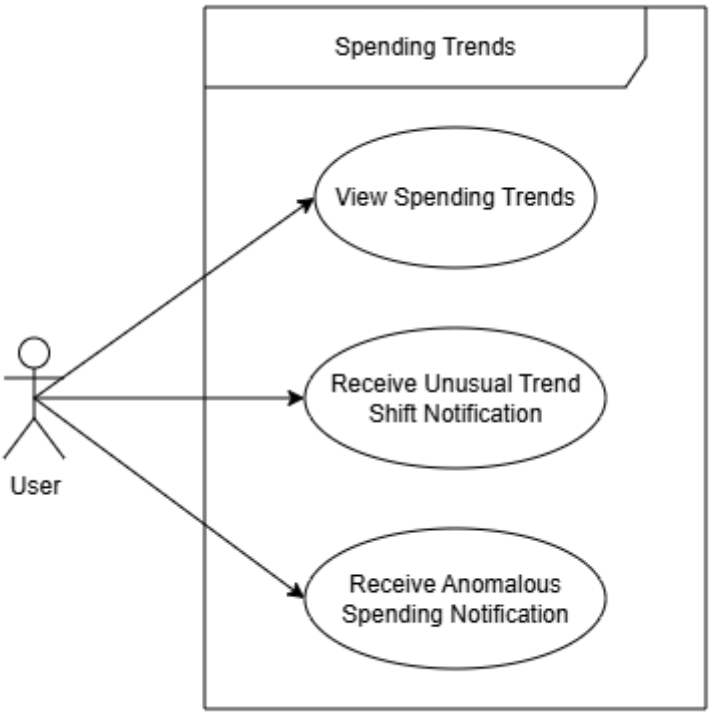

Notifications
Select Notification Platform
Receive Notifications through Selected Platform
User


Budgets
Define Budget Thershold
Receive Overspending Alert
Receive Near-Overspending Alert
User


Comparisons
Compare Cost across Cloud Providers
Compare Provider Views
View Comparison Summaries
User


Dashboard Summaries
View Usage Summary
<<extend>>
View Metric Summaries
View Spend Summary
<<extend>>
User
View Analytics KPIs


Reports
<<extend>>
Download Cloud Cost Reports
Download Reports
Download Trend Analysis Reports
<<extend>>
<<extend>>
View Trend Analysis Reports
View Reports
<<extend>>
View Cloud Cost Reports
User
<<extend>>
Filter by Provider
Filter Reports
<<extend>>
Filter by Time Period
Filter by Environment
<<extend>>



5 Functional Requirements

5.1 Cloud Data Integration

Description

CloudSherpa must connect to supported cloud providers and acquire the operational data needed for analysis.

Requirement Statement

The system shall integrate with cloud providers (**AWS**, **Azure**, and **GCP**) to collect:

- Usage metrics
- Billing data
- Performance data

Purpose

This provides the raw data foundation required for all downstream analytics, forecasting, anomaly detection, and optimization.

Subsystem Allocation

Ingestion Service (Connection Manager and Cloud Connector Modules)

5.2 Data Storage and Processing

Description

CloudSherpa must persist collected data and prepare it for analytical workloads.

Requirement Statement

The system shall:

- Store historical cloud data
- Process and aggregate time-series data for analytics

Purpose

Historical and aggregated data are necessary for trend analysis, reporting, and AI-driven predictions.

Subsystem Allocation

Ingestion Service (Normalization module) and Persistence Service

---

5.3 Cost Prediction

Description

CloudSherpa must provide forward-looking insight into cloud expenditure.

Requirement Statement

The system shall apply machine learning models to:

- Forecast future cloud costs

- Identify spending trends

Purpose

Forecasting supports proactive budgeting and helps users anticipate future cloud-spend patterns.

Subsystem Allocation

Application Service (Forecast module)

5.4 Optimization Recommendations

Description

CloudSherpa must identify inefficient infrastructure utilization and recommend corrective actions.

Requirement Statement

The system shall detect inefficiencies such as:

- Idle resources
- Overprovisioned resources

The system shall also provide cost-saving recommendations.

Purpose

This enables users to reduce waste and improve financial efficiency across their cloud environments.

Subsystem Allocation

Application Service (Optimization module)

5.5 Web-Based Dashboard

Description

CloudSherpa must present relevant cloud and financial data through a unified web interface.

Requirement Statement

The system shall provide a web interface to:

- Visualize usage metrics
- Display forecasts
- Present recommendations

Purpose

A centralized and intuitive dashboard makes insights accessible to both technical and non-technical stakeholders.

Subsystem Allocation

Presentation Service (Dashboard module)

5.6 User and Account Management

Description

CloudSherpa must manage user identities, cloud-account connections, and access permissions.

Requirement Statement

The system shall allow users to:

- Connect multiple cloud accounts
- Manage environments
- Control access permissions

Purpose

Secure and structured access management is essential for a platform dealing with sensitive operational and billing data.

Subsystem Allocation

Application Service (Authentication module)

5.7 Alerts and Notifications

Description

CloudSherpa should proactively notify users about important cloud-cost events and opportunities.

Requirement Statement

The system should:

- Notify users of anomalies
- Alert on budget thresholds
- Send notifications via email or SMS

Purpose

Proactive communication improves response time and reduces the likelihood of unnoticed overspending.

Subsystem Allocation

Application Service (Anomaly and Alert modules)

5.8 Multi-Cloud Cost Comparison

Description

CloudSherpa must support direct comparison across supported cloud providers.

Requirement Statement

The system shall compare costs and usage across providers.

Purpose

Cross-provider comparison helps users identify the most cost-effective environment for workloads and infrastructure decisions.

Subsystem Allocation

Application Service (Analytics and optimization modules)

6 Quality Requirements

6.1 Scalability

Description

CloudSherpa must remain effective as data volume, user count, and provider integrations grow.

Requirement Statement

The system shall:

- Support large-scale cloud data across multiple providers
- Support independent scaling of components

Purpose

A multi-cloud analytics platform must accommodate increasing ingestion and processing loads without requiring a redesign.

6.2 Performance

Description

CloudSherpa must deliver timely updates and responsive user interactions.

Requirement Statement

The system shall:

- Provide near real-time data ingestion and processing
- Ensure dashboard updates are responsive and timely

Purpose

Timely feedback is essential for anomaly response, cost visibility, and practical operational use.

6.3 Security

Description

CloudSherpa must protect sensitive user, account, and cloud-provider information.

Requirement Statement

The system shall:

- Securely handle cloud credentials
- Encrypt sensitive data
- Enforce access control mechanisms

Purpose

Security is critical because the system interacts with cloud accounts, billing data, and potentially privileged operational metadata.

6.4 Reliability and Availability

Description

CloudSherpa must remain dependable in the face of operational failures.

Requirement Statement

The system shall:

- Ensure high availability of services
- Handle failures gracefully

Purpose

Users must be able to trust the platform's output and availability, particularly for monitoring and decision support.

6.5 Usability

Description

CloudSherpa must be accessible to users with varying levels of technical expertise.

Requirement Statement

The user interface shall be:

- Intuitive
- Easy to navigate
- Suitable for users with varying technical expertise

Purpose

A usable interface broadens adoption and ensures business value is not limited to deeply technical users.

6.6 Maintainability

Description

CloudSherpa must be structured in a way that supports ongoing development and change.

Requirement Statement

The system shall follow a modular architecture where:

- Each module manages its own data
- Communication occurs through well-defined APIs

Purpose

Maintainability reduces long-term development overhead and supports safe evolution of the platform.

6.7 Interoperability

Description

CloudSherpa must successfully operate across its targeted external environments.

Requirement Statement

The system shall integrate with:

- AWS
- Azure
- GCP

Other providers are out of scope.

Purpose

Interoperability with the intended providers is central to the multi-cloud value proposition of CloudSherpa.

6.8 Extensibility

Description

CloudSherpa should be capable of future expansion without major architectural upheaval.

Requirement Statement

The system should support future enhancements such as:

- Additional providers
- Advanced AI capabilities

Purpose

Even though some features are initially out of scope, the architecture should not block their future introduction.

6.9 Data Consistency and Integrity

Description

CloudSherpa must maintain trustworthy and accurate data throughout ingestion, processing, and presentation.

Requirement Statement

The system shall ensure the correctness and consistency of collected and processed data.

Purpose

Inaccurate or inconsistent data would undermine recommendations, forecasts, and trust in the platform.

6.10 Compliance and Best Practices

Description

CloudSherpa must align with accepted standards for secure and well-structured system design.

Requirement Statement

The system shall adhere to industry best practices for:

- Cloud security
- Data handling
- API design

Purpose

Standards-aligned design improves quality, reduces risk, and supports professional maintainability.

7 API Service Contracts

7.1 Ingestion

`/api/events/ingest`

Request:

```
{
  "scopes": [
    {
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceScopes": [
        {
          "name": "string",
          "instances": [
            {
              "identifierName": "string",
              "values": [
                "string"
              ]
            }
          ],
          "metrics": [
            "string"
          ]
        }
      ]
    }
  ],
  "credentials": {
    "accessKey": "string",
    "secretKey": "string",
    "tenantId": "string",
    "clientId": "string",
    "clientSecret": "string",
    "projectId": "string"
  },
  "from": "2026-05-21T18:44:09.423Z",
  "to": "2026-05-21T18:44:09.423Z",
  "period": 0,
  "includeBilling": true,
```

```
  "includeUsage": true,
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Response:

```
{
  "usage": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "resourceId": "string",
      "resourceType": "string",
      "serviceName": "string",
      "region": "string",
      "metricName": "string",
      "value": 0.1,
      "unit": "string",
      "timestamp": "2026-05-21T18:44:09.435Z",
      "periodStart": "2026-05-21T18:44:09.435Z",
      "periodEnd": "2026-05-21T18:44:09.435Z",
      "dimensions": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-05-21T18:44:09.435Z",
      "ingestionId": "string",
      "source": "string"
    }
  ],
  "billing": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
```

```
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceName": "string",
      "resourceId": "string",
      "resourceType": "string",
      "region": "string",
      "cost": 0.1,
      "usageQuantity": 0.1,
      "unit": "string",
      "currency": "string",
      "pricingModel": "string",
      "usageStartTime": "2026-05-21T18:44:09.435Z",
      "usageEndTime": "2026-05-21T18:44:09.435Z",
      "billingPeriodStart": "2026-05-21T18:44:09.435Z",
      "billingPeriodEnd": "2026-05-21T18:44:09.435Z",
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-05-21T18:44:09.435Z",
      "ingestionId": "string",
      "source": "string"
    }
  ]
}
```

/api/events/ingest/mock

Request:

```
{
  "scopes": [
    {
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceScopes": [
        {
```

```
          "name": "string",
          "instances": [
            {
              "identifierName": "string",
              "values": [
                "string"
              ]
            }
          ],
          "metrics": [
            "string"
          ]
        }
      ]
    }
  ],
  "credentials": {
    "accessKey": "string",
    "secretKey": "string",
    "tenantId": "string",
    "clientId": "string",
    "clientSecret": "string",
    "projectId": "string"
  },
  "from": "2026-05-21T18:45:51.935Z",
  "to": "2026-05-21T18:45:51.935Z",
  "period": 0,
  "includeBilling": true,
  "includeUsage": true,
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Response:

```
{
  "usage": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "resourceId": "string",
```

```
      "resourceType": "string",
      "serviceName": "string",
      "region": "string",
      "metricName": "string",
      "value": 0.1,
      "unit": "string",
      "timestamp": "2026-05-21T18:45:51.937Z",
      "periodStart": "2026-05-21T18:45:51.937Z",
      "periodEnd": "2026-05-21T18:45:51.937Z",
      "dimensions": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-05-21T18:45:51.937Z",
      "ingestionId": "string",
      "source": "string"
    }
  ],
  "billing": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceName": "string",
      "resourceId": "string",
      "resourceType": "string",
      "region": "string",
      "cost": 0.1,
      "usageQuantity": 0.1,
      "unit": "string",
      "currency": "string",
      "pricingModel": "string",
      "usageStartTime": "2026-05-21T18:45:51.937Z",
      "usageEndTime": "2026-05-21T18:45:51.937Z",
      "billingPeriodStart": "2026-05-21T18:45:51.937Z",
```

```
      "billingPeriodEnd": "2026-05-21T18:45:51.937Z",
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-05-21T18:45:51.937Z",
      "ingestionId": "string",
      "source": "string"
    }
  ]
}
```

/api/events/ingest/mockNoise

Request:

```
{
  "scopes": [
    {
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceScopes": [
        {
          "name": "string",
          "instances": [
            {
              "identifierName": "string",
              "values": [
                "string"
              ]
            }
          ],
          "metrics": [
            "string"
          ]
        }
      ]
    }
  ],
```

```
  "credentials": {
    "accessKey": "string",
    "secretKey": "string",
    "tenantId": "string",
    "clientId": "string",
    "clientSecret": "string",
    "projectId": "string"
  },
  "from": "2026-05-21T18:46:55.739Z",
  "to": "2026-05-21T18:46:55.739Z",
  "period": 0,
  "includeBilling": true,
  "includeUsage": true,
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Response:

```
{
  "usage": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "resourceId": "string",
      "resourceType": "string",
      "serviceName": "string",
      "region": "string",
      "metricName": "string",
      "value": 0.1,
      "unit": "string",
      "timestamp": "2026-05-21T18:46:55.742Z",
      "periodStart": "2026-05-21T18:46:55.742Z",
      "periodEnd": "2026-05-21T18:46:55.742Z",
      "dimensions": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "tags": {
        "additionalProp1": "string",
```

```
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-05-21T18:46:55.742Z",
      "ingestionId": "string",
      "source": "string"
    }
  ],
  "billing": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceName": "string",
      "resourceId": "string",
      "resourceType": "string",
      "region": "string",
      "cost": 0.1,
      "usageQuantity": 0.1,
      "unit": "string",
      "currency": "string",
      "pricingModel": "string",
      "usageStartTime": "2026-05-21T18:46:55.742Z",
      "usageEndTime": "2026-05-21T18:46:55.742Z",
      "billingPeriodStart": "2026-05-21T18:46:55.742Z",
      "billingPeriodEnd": "2026-05-21T18:46:55.742Z",
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-05-21T18:46:55.742Z",
      "ingestionId": "string",
      "source": "string"
    }
  ]
}
```


7.2 Authentication

/auth/register

Request:

```
{
  "email": "string",
  "username": "string",
  "password": "string"
}
```

Response:

```
{
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "email": "string",
  "username": "string",
  "token": "string"
}
```

/auth/login

Request:

```
{
  "email": "string",
  "password": "string"
}
```

Response:

```
{
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "email": "string",
  "username": "string",
  "token": "string"
}
```

7.3 Analytics

/analytics/resource-names

Request: *No parameters.*

Response:

```
{
  "additionalProp1": "string",
  "additionalProp2": "string",
  "additionalProp3": "string"
}
```

/analytics/historical

Request Parameters:

- `from` (*string*)
- `to` (*string*)

Response:

```
[
  {
    "metricId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "accountId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "resourceId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "recordedAt": "2026-05-21T18:50:58.620Z",
    "metricType": "string",
    "metricName": "string",
    "metricValue": 0,
    "unit": "string",
    "currency": "string",
    "periodStart": "2026-05-21T18:50:58.620Z",
    "periodEnd": "2026-05-21T18:50:58.620Z"
  }
]
```

7.4 SSE (Server-Sent Events)

/stream

Request: *No parameters.*

Response:

```
{
  "timeout": 0
}
```

8 Constraints

8.1 Provider Scope Constraint

Scope

CloudSherpa is limited to a defined provider set.

Requirement Statement

The system is limited to:

- AWS
- Azure
- GCP

Additional providers are out of scope.

8.2 Security Constraint

Scope

Sensitive integration and user data must be treated securely.

Requirement Statement

The system should ensure secure handling of cloud credentials and user data, following best practices for encryption and access control.

8.3 Usability Constraint

Scope

The platform must remain accessible to a broad user base.

Requirement Statement

The user interface should be intuitive and easy to navigate, suitable for users with varying levels of technical expertise.

8.4 Budget Constraint

Scope

Development and deployment choices must remain cost-conscious.

Requirement Statement

As far as possible, the system should use free resources. If needed, funds can be discussed.